Exp No: 6	Error Correction at Data Link Layer using Hamming Code

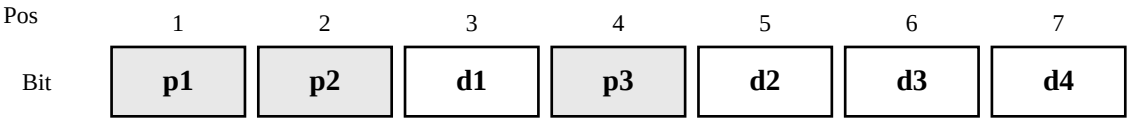
AIM:

To write a program that detects and corrects a single-bit error in a data stream using the Hamming (7,4) code, and to verify the error correction feature with a test run.

ALGORITHM / PROCEDURE:

1. Take the 4 data bits d1 d2 d3 d4 as input from the user.
2. Place the bits in a 7-bit codeword. Positions 1, 2 and 4 (powers of 2) are kept for the parity bits p1, p2, p3 and the rest hold the data bits.
3. Calculate the even parity bits: p1 = d1 ^ d2 ^ d4, p2 = d1 ^ d3 ^ d4, p3 = d2 ^ d3 ^ d4.
4. Send the 7-bit codeword. Flip one bit (by user input) to act as a transmission error.
5. At the receiver, recompute the three parity checks s1, s2 and s3 using the received bits.
6. The binary number s3 s2 s1 is the syndrome. If it is 000 there is no error, otherwise it is the position of the wrong bit.
7. Flip the bit at that position to correct it and extract the original data bits.
8. Compare the recovered data with the original data and display the result.

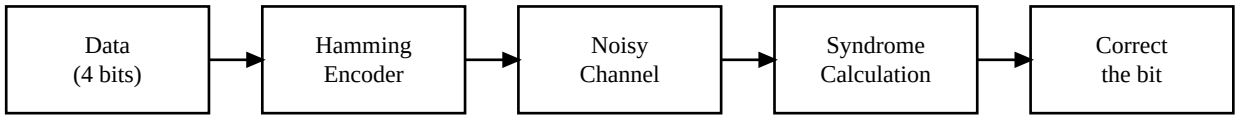
Fig 1: Hamming (7,4) codeword layout



p1 checks positions 1,3,5,7 p2 checks 2,3,6,7 p3 checks 4,5,6,7

Shaded boxes = parity bits (positions 1, 2, 4). Unshaded = data bits.

Fig 2: Block diagram of error detection and correction



PROGRAM:

```
# Hamming (7,4) error detection and correction
def encode(d):                      # d = [d1, d2, d3, d4]
    d1, d2, d3, d4 = d
    p1 = d1 ^ d2 ^ d4
    p2 = d1 ^ d3 ^ d4
    p3 = d2 ^ d3 ^ d4
    return [p1, p2, d1, p3, d2, d3, d4]   # positions 1..7

def decode(c):
    s1 = c[0] ^ c[2] ^ c[4] ^ c[6]
    s2 = c[1] ^ c[2] ^ c[5] ^ c[6]
    s3 = c[3] ^ c[4] ^ c[5] ^ c[6]
    pos = s3 * 4 + s2 * 2 + s1          # 0 means no error
    if pos != 0:
        c[pos - 1] ^= 1                 # correct the bit
    return pos, [c[2], c[4], c[5], c[6]]

data = [int(b) for b in input("Enter 4 data bits: ")]
code = encode(data)
print("Data bits        :", *data)
print("Hamming codeword :", *code)

recv = code[:]
p = int(input("Enter bit position (1-7) to corrupt, 0 for none: "))
if p:
    recv[p - 1] ^= 1
print("Received word    :", *recv)

pos, corrected = decode(recv)
if pos == 0:
    print("No error detected.")
else:
    print("Error detected at position", pos)
    print("Corrected word   :", *recv)
print("Recovered data   :", *corrected)
print("Verification     :", "SUCCESS" if corrected == data else "FAILED")
```

OUTPUT:

Terminal - Test run 1 (error in bit 6)

```
USER   @USER   -VirtualBox:~$ python3 exp6.py
Enter 4 data bits: 1011
Data bits        : 1 0 1 1
Hamming codeword : 0 1 1 0 0 1 1
Enter bit position (1-7) to corrupt, 0 for none: 6
Received word    : 0 1 1 0 0 0 1
Error detected at position 6
Corrected word   : 0 1 1 0 0 1 1
Recovered data   : 1 0 1 1
Verification     : SUCCESS
```

Terminal - Test run 2 (no error)

```
USER   @USER   -VirtualBox:~$ python3 exp6.py
Enter 4 data bits: 0110
Data bits        : 0 1 1 0
Hamming codeword : 1 1 0 0 1 1 0
Enter bit position (1-7) to corrupt, 0 for none: 0
Received word    : 1 1 0 0 1 1 0
No error detected.
Recovered data   : 0 1 1 0
Verification     : SUCCESS
```

Terminal - Test run 3 (error in bit 3)

```
USER   @USER   -VirtualBox:~$ python3 exp6.py
Enter 4 data bits: 1101
Data bits        : 1 1 0 1
Hamming codeword : 1 0 1 0 1 0 1
Enter bit position (1-7) to corrupt, 0 for none: 3
Received word    : 1 0 0 0 1 0 1
Error detected at position 3
Corrected word   : 1 0 1 0 1 0 1
Recovered data   : 1 1 0 1
Verification     : SUCCESS
```

RESULT:

Thus the program for error detection and correction using Hamming code was written and executed successfully. A single-bit error was found at the correct position and corrected in every test run.